

Task 1:

TASK 1: Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficient Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>
#include <string.h>

#define DATA_SIZE 100000

int main()
{
    int pipefd[2];
    pid_t pid;
    char buffer[1024];
    char data[] = "Producer sends data to Consumer through pipe.\n";
    long total_bytes = 0;

    struct timespec start, end;

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(1);
    }

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        exit(1);
    }

    if (pid == 0)
    {
        // Child - Consumer
        close(pipefd[1]);

        while (1)
        {
            int n = read(pipefd[0], buffer, sizeof(buffer));

            if (n <= 0)
                break;

            total_bytes += n;
        }

        close(pipefd[0]);

        printf("Consumer received %ld bytes.\n", total_bytes);
        exit(0);
    }
    else
    {
        // Parent - Producer
        close(pipefd[0]);

        clock_gettime(CLOCK_MONOTONIC, &start);

        for (int i = 0; i < DATA_SIZE; i++)
        {
            write(pipefd[1], data, sizeof(data));
        }
    }
}
```

```

}

if (pid == 0)
{
    // Child - Consumer
    close(pipefd[1]);

    while (1)
    {
        int n = read(pipefd[0], buffer, sizeof(buffer));

        if (n <= 0)
            break;

        total_bytes += n;
    }

    close(pipefd[0]);

    printf("Consumer received %ld bytes.\n", total_bytes);
    exit(0);
}
else
{
    // Parent - Producer
    close(pipefd[0]);

    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int i = 0; i < DATA_SIZE; i++)
    {
        write(pipefd[1], data, strlen(data));
    }

    close(pipefd[1]);

    wait(NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_taken =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1e9;

    long bytes_sent = DATA_SIZE * strlen(data);

    printf("Producer sent %ld bytes.\n", bytes_sent);
    printf("Communication time: %.6f seconds\n", time_taken);

    if (time_taken > 0)
    {
        double throughput =
            (bytes_sent / time_taken) / (1024 * 1024);

        printf("Communication throughput: %.2f MB/s\n",
            throughput);
    }
}

return 0;
}

```

Output:

```

sirichandana@siri:~$ gcc w5.c -o w5
sirichandana@siri:~$ ./w5
Consumer received 4600000 bytes.
Producer sent 4600000 bytes.
Communication time: 0.147341 seconds
Communication throughput: 29.77 MB/s
sirichandana@siri:~$

```

Observation:

The **parent process produces data and writes it into the anonymous pipe**, while the **child process reads and consumes the data**. Communication efficiency is measured using the **time taken and data throughput (MB/s)**.

Task 2:

Develop a program that executes the equivalent of the shell command: `ls -l grep` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    // Create pipe
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(1);
    }

    // Create first child for ls -l
    pid1 = fork();

    if (pid1 == -1)
    {
        perror("fork");
        exit(1);
    }

    if (pid1 == 0)
    {
        // Redirect stdout to pipe
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);

        // Execute ls -l
        execlp("ls", "ls", "-l", (char *)NULL);

        perror("execlp ls");
        exit(1);
    }

    // Create second child for grep
    pid2 = fork();

    if (pid2 == -1)
    {
        perror("fork");
        exit(1);
    }

    if (pid2 == 0)
    {
        // Redirect stdin from pipe
        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);

        // Execute grep
        execlp("grep", "grep", ".", (char *)NULL);

        perror("execlp grep");
    }
}

```

```

        perror("fork");
        exit(1);
    }

    if (pid2 == 0)
    {
        // Redirect stdin from pipe
        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);

        // Execute grep
        execlp("grep", "grep", ".", (char *)NULL);

        perror("execlp grep");
        exit(1);
    }

    // Parent closes both ends of pipe
    close(pipefd[0]);
    close(pipefd[1]);

    // Wait for both children
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("Parent: Both processes completed.\n");

    return 0;
}

```

Output:

```

sirichandana@siri:~$ ls -l
total 268
-rw-r--r-- 1 sirichandana siri 57 Aug 6 14:47 First.c
drwxr-xr-x 3 sirichandana siri 4096 Aug 3 05:02 Hello-World
drwxr-xr-x 11 sirichandana siri 4096 Jul 30 09:39 OSPP
-rw-r--r-- 1 sirichandana siri 1322 Aug 25 03:28 Pweek2.c
-rw-r--r-- 1 sirichandana siri 1206 Aug 7 15:18 Practical2.c
-rw-r--r-- 1 sirichandana siri 935 Aug 6 15:00 Skill2.c
-rw-r--r-- 1 sirichandana siri 664 Aug 7 14:32 Skill3.c
-rw-r--r-- 1 sirichandana siri 802 Aug 7 14:48 Skill3Task2.c
-rwxr-xr-x 1 sirichandana siri 16356 Aug 27 09:58 a.out
-rw-r--r-- 1 sirichandana siri 936 Aug 5 16:22 command_exec.c
-rw-r--r-- 1 sirichandana siri 0 Aug 25 04:47 copy.txt
-rw-r--r-- 1 sirichandana siri 24 Aug 25 04:56 destination.txt
-rw-r--r-- 1 sirichandana siri 1 Jul 28 14:48 nano,565.save
-rw-r--r-- 1 sirichandana siri 729 Aug 25 04:43 new.c
-rw-r--r-- 1 sirichandana siri 270 Aug 25 04:46 new1.c
-rw-r-xr-x 1 sirichandana siri 16216 Aug 25 04:50 new12
-rw-r--r-- 1 sirichandana siri 1252 Aug 25 04:55 new12.c
-rw-r--r-- 1 sirichandana siri 710 Aug 25 04:42 os-p-2.c
-rw-r--r-- 1 sirichandana siri 931 Aug 20 08:37 os.c
-rw-r--r-- 1 sirichandana siri 156 Aug 18 07:00 os1
drwxr-xr-x 6 sirichandana siri 4096 Aug 4 08:12 ossp
-rw-r--r-- 1 sirichandana siri 148 Aug 19 09:50 si.c
-rw-r--r-- 1 sirichandana siri 881 Aug 19 23:47 s12.c
-rw-r--r-- 1 sirichandana siri 538 Aug 20 00:00 s21.c
-rwxr-xr-x 1 sirichandana siri 585 Aug 25 05:01 sample.txt
-rw-r--r-- 1 sirichandana siri 3735 Aug 22 06:33 sk1.c
-rwxr-xr-x 1 sirichandana siri 1044 Aug 11 09:48 skill4Task1.c
-rw-r--r-- 1 sirichandana siri 5928 Aug 27 09:52 skill3Task2.c
-rwxr-xr-x 1 sirichandana siri 1336 Aug 27 09:49 skillwe2k2.c
-rw-r--r-- 1 sirichandana siri 4192 Aug 24 06:12 weillwek2.c
-rw-r--r-- 1 sirichandana siri 5926 Aug 24 13:58 skillweek2.c
-rw-r--r-- 1 sirichandana siri 920 Aug 27 08:09 weeeekTask4.c
-rwxr-xr-x 1 sirichandana siri 16256 Aug 27 08:37 week42.c
-rw-r-xr-x 1 sirichandana siri 18113 Aug 22 06:19 week42.save
-rw-r-xr-x 1 sirichandana siri 926 Aug 27 08:14 week41.c

```

Observation:

- The program successfully implements the `ls -l | grep .` command using `fork()`, `pipe()`, `dup2()`, and `exec()`.
- It demonstrates **inter-process communication**, where the output of `ls` is passed through the pipe to `grep`.